

Programmation Orientée Objet et C++

Contrôle Terminal

10 janvier 2023

Durée : 3h

Tous les documents disponibles sur Moodle sont autorisés.

Accès à Moodle : <http://cours.univ-perp.fr>

Consignes importantes à lire avant de commencer :

1. Votre rendu sera composé de :
 - votre copie papier contenant les réponses des questions de la première partie.
 - d'une archive **nom-prénom.zip** que vous déposerez sur le Moodle du cours avant la fin de l'épreuve.
 2. Votre archive comprendra :
 - un fichier **Makefile**
 - un fichier **main.cpp** contenant le corps principal de votre programme.
 - un dossier **src/** contenant les fichiers sources (**.cpp**)
 - un dossier **include** contenant les fichiers d'en-têtes (**.hpp**)
 - un dossier **obj/** dans lequel devront être créés les différents fichiers objets (**.o**) lors de la compilation.
 - un dossier **data/** qui permettra de sauvegarder/lire des données pour le programme.
 - Vous complétez votre **Makefile** au fur et à mesure de votre progression. L'absence de **Makefile** sera **PÉNALISÉE**.
 3. La qualité et la clarté de la programmation seront prises en compte. Le code rendu devra pouvoir **COMPILER** et le programme s'exécuter sans **ERREUR** et sans **FUITES MÉMOIRES**. Les codes qui ne compileront pas ne seront pas pris en compte dans la notation.
-

1 Questions de Programmation C++ (5pts)

Vous répondrez aux questions de cette partie sur votre copie papier.

1. Que se passe-t-il à l'exécution du code suivant, pourquoi et comment régler le problème ?

```
1    class A {
2        private:
3            int *a;
4            int size;
5
6        public:
7            A(){}
8            A(int s): size(s) {a = new int[size];}
9            ~A() {delete [] a;}
10    };
11
12    int main() {
13        A a(10), b;
14        b = a;
```

```

15
16     return 0;
17 }

```

2. Donnez la valeur des attributs de obj à la ligne 21.

```

1     struct A {
2         int a;
3         A(){}
4         A(int v): a(v) {}
5     };
6
7     struct B {
8         int b;
9         B(){}
10        B(int v): b(v) {}
11    };
12
13    struct C: A, B {
14        int c;
15        C(){}
16        C(int v=0): c(v) {}
17    };
18
19    int main() {
20        C obj(7);
21
22        return 0;
23    }

```

3. Que se passera-t-il à l'exécution du code suivant ? Comment changer l'affichage du `print` à la ligne 21 ?

```

1     #include <iostream>
2
3     class A {
4     public:
5         void print(){std::cout << "Je suis un objet de type A" << std::endl;}
6     };
7
8     class B : public A{
9     public:
10        void print(){std::cout << "Je suis un objet de type B" << std::endl;}
11    };
12
13    int main() {
14        A a; B b;
15
16        a.print();
17        a = b;
18        a.print();
19
20        A &ref = b;
21        ref.print();
22
23        return 0;
24    }

```

2 Simulateur de circuits logiques (15 pts)

L'objectif de ce travail est de créer un programme capable de simuler l'exécution d'un circuit logique simple et d'en obtenir la table de vérité.

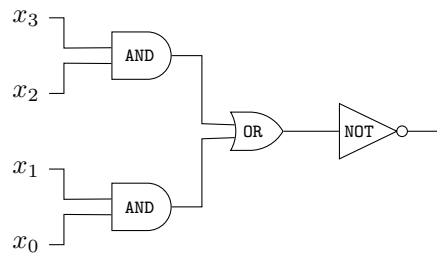


Figure 1: Exemple de circuit logique avec 4 bits en entrée.

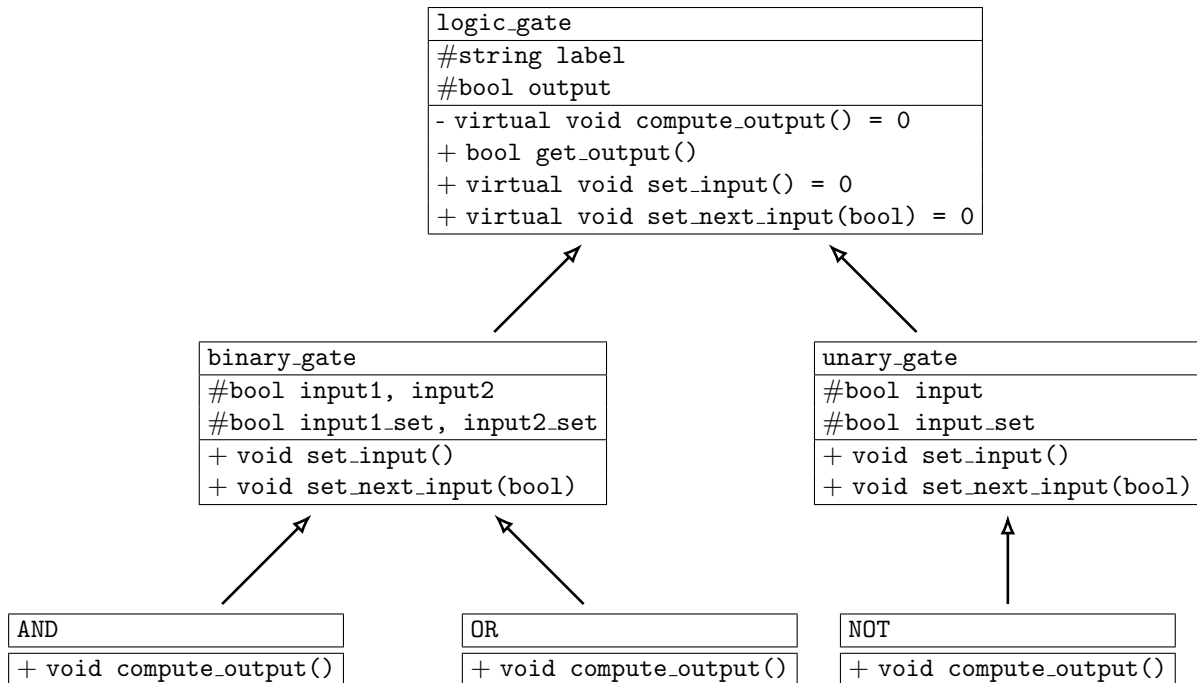


Figure 2: Diagramme UML des classes portes logiques

2.1 Porte logique

La brique de base de notre simulateur de circuit va être une classe `logic_gate` qui va permettre de créer des portes logiques. Dans notre cas nous nous limiterons à des portes logiques n'ayant qu'un seul bit de sortie.

Nous allons considérer deux types de portes logiques : les portes *unaires* et les portes *binaires*. Les portes binaires reçoivent deux bits en entrée tandis que les portes unaires ne reçoivent qu'un seul bit en entrée.

Pour notre simulateur nous nous contenterons d'implémenter deux portes binaires : OR et AND et une seule porte unaire NOT. Les tables de vérité de ces trois portes sont rappelées ci-dessous :

OR	0	1
0	0	1
1	1	1

AND	0	1
0	0	0
1	0	1

NOT	0	1
0	1	0
1	0	1

1. Créez un module (fichier d'en-tête et fichier source) `gates` permettant d'implémenter les portes logiques selon la logique présentée dans la Figure 2. Vous êtes libre de rajouter des méthodes et attributs aux classes présentées dans cette figure.

La méthode `void set_next_input(bool)` initialise le premier bit en entrée d'une porte, et si celui-ci est déjà initialisé (variable `input_set = true`) alors elle initialise le second.

2.2 Circuits logiques

Nous allons maintenant pouvoir construire des circuits logiques. Un circuit est composé de plusieurs portes logiques. Parmi ces portes logiques, il faudra distinguer les portes en entrée du circuit (celles dont les entrées

ne sont connectées à aucune porte logique) des autres.

Dans un circuit à n portes (indexées de 0 à $n - 1$), les k portes en entrée ($k < n$) seront celles numérotées de 0 à $k - 1$.

On disposera d'un tableau `tab` de n éléments permettant d'indiquer les connexions entre les différentes portes. L'affectation `tab[i]=j` permettra d'indiquer que la sortie de la porte d'indice i est connectée à une entrée de la porte d'indice j . Si la sortie d'une porte d'indice i n'est connectée à aucune autre porte, on indiquera `tab[i]=-1`.

On initialisera les circuits logiques à partir d'un fichier de description. Le fichier décrivant le circuit de la Figure 1 est donné ci-dessous et est disponible sur Moodle sous le nom `circuit_figure_1.txt` :

```
4 <-- nombre de portes dans le circuit (n)

AND <-- type de la porte 0 (entree)
AND <-- type de la porte 1 (entree)
OR <-- type de la porte 2
NOT <-- type de la porte 3

2 <-- nombre de portes en entree (k)

2 <-- tab[0]
2 <-- tab[1]
3 <-- tab[2]
-1 <-- tab[3]
```

2. Créez un module `circuit` qui implémentera une classe `circuit` respectant les spécifications suivantes :

- un circuit sera construit à partir d'un fichier.
- on pourra initialiser les bits en entrées du circuit en les saisissant au clavier
- on pourra afficher la table de vérité du circuit

Attention : un circuit peut avoir plus d'un bit en sortie !

```
./main

Circuit of file circuit_figure_1.txt loaded.

Gate AND, enter input1 : 1
Gate AND, enter input2 : 1
Gate AND, enter input1 : 0
Gate AND, enter input2 : 0

Result : 0

Truth table of the circuit

| 0 0 0 0 | 1 |
| 0 0 0 1 | 1 |
| 0 0 1 0 | 1 |
| 0 0 1 1 | 0 |
| 0 1 0 0 | 1 |
| 0 1 0 1 | 1 |
| 0 1 1 0 | 1 |
| 0 1 1 1 | 0 |
| 1 0 0 0 | 1 |
| 1 0 0 1 | 1 |
| 1 0 1 0 | 1 |
| 1 0 1 1 | 0 |
| 1 1 0 0 | 0 |
| 1 1 0 1 | 0 |
| 1 1 1 0 | 0 |
| 1 1 1 1 | 0 |
```

Figure 3: Exemple d'exécution du programme sur le circuit de la Figure 1